

Byte-Interval Mediation: Read-Receipt Write Admission for Concurrent LLM Coding Agents

Joshua Pinckard
ToolsEnabled
josh@toolsenabled.ai

September 2026

Abstract

When several autonomous coding agents edit one working tree, each writes from its own view of files that others may have changed. The result is the lost update and the stale read of the database literature, arising in plain files with no transaction manager. We describe byte-interval mediation, the file-level concurrency control implemented as the byte authority of the ToolsEnabled engine. The authority records a durable receipt for every mediated read, binding a transport scope, a file, an exact half-open byte interval, content and file hashes, a version and a durable sequence number. It admits a patch only when the patching scope observed the changed span and its observations on every file pass admission-time freshness checks. Length-changing edits rebase byte-disjoint observations and invalidate overlapping ones. An unmediated change forces a whole-file reread, and a whole-file write is recorded as blind replacement. Creation is exclusive, and every publication is journaled as PREPARED and then COMMITTED, so that after a crash each pending publication resolves from the bytes actually present. The implementation passes its 68 unit, recovery and multi-process tests in a standalone extraction, and 117 tests including transport integration in the engine. Six design probes pin its limits. The most important is that byte-disjoint edits can jointly break a shared invariant. In the inspected deployment, the agents' coding workflow bypassed the authority through unmediated host file tools; this observation does not qualify a final release.

1 Introduction

Coding agents built on large language models now resolve repository issues end to end [11, 33, 35–37], and frameworks run several of them together [10, 25, 34]. When agents share one working tree, each edits from its own view of files that others may have changed since. Two failures follow. In a *lost update*, agent B overwrites a region that agent A changed after B read it. In a *stale read*, A's edit is correct against bytes that no longer exist, possibly in another file. Both are classic isolation anomalies [1, 2]. Here they occur in plain files, with no transaction manager, between a read and a write that can be minutes apart while a model reasons.

This paper describes byte-interval mediation, implemented as the byte authority of the ToolsEnabled engine. Its contributions are:

1. a write-admission rule for files based on durable read receipts over exact byte intervals: a patch publishes only if its scope observed the changed span and everything the scope read is still current (Section 3);
2. coordinate maintenance under length-changing edits, a whole-file-reread rule for unmediated changes, and an explicit class for blind whole-file writes (Section 3);
3. a publication journal with exclusive, identity-checked creation, and recovery that resolves each pending publication from the bytes on disk (Sections 3 and 4);
4. a standalone extraction with the unchanged implementation and suites, six design probes that pin its limits, and an account of a deployment in which it was dormant (Sections 6 and 7).

The rule does not address semantic conflicts. Two byte-disjoint edits can jointly break an invariant, and the authority admits both (probe P4, Section 6).

2 Setting

A host runs several agents. Each agent reaches files only through tools, and the host gives each authenticated tool session a private *scope* that tool arguments cannot name or forge. An agent reads parts of files, reasons, and then either patches a file (replacing one exact span) or writes a whole file. Nothing orders these steps across agents.

Three design choices shape the mechanism; they are design statements, not measured results. *Validate, do not lock.* Holding locks across model inference would serialize agents for minutes and require timeouts. Instead the read set is accumulated from the reads an agent actually makes and validated when it writes, as in optimistic concurrency control [15]. Observations expire, as leases do [9]. *Refuse, do not merge.* Merging concurrent edits yields a converged file but does not tell an agent that its premise changed. A refused write returns the regions to reread, and the agent reconciles. *Bytes, not lines.* Offsets count bytes and hashes cover the exact materialized bytes, so line-ending and encoding differences cannot hide a change, and no parser is needed for any language.

3 Design

Receipts and observations. A *resource* is a canonical absolute path to a file of at most 512 KiB. The authority records its version, SHA-256, length and presence; a missing file and an empty file are distinct states. A read of the half-open interval $[a, b)$ materializes the whole file through an adapter, validates the window (the engine refuses windows that are not complete UTF-8) and, before releasing any content, durably records a *receipt*: the scope, the resource, $[a, b)$, the SHA-256 of the returned bytes and of the whole file, the resource version, a lease expiry and the sequence number of an append-only event. The live projection of a receipt is one or more *observations*: intervals with their observed bytes and an optional stale reason. A new read replaces the scope’s older observations that it intersects or covers, keeping their parts outside the new window.

Conflict is interval intersection. Non-empty intervals $[a, b)$ and $[c, d)$ conflict when $a < d$ and $c < b$, so adjacent intervals do not conflict. A point p (an insertion, or an observation of an empty file) conflicts with $[c, d)$ when $c \leq p < d$: inserting at the left edge of another scope’s observation conflicts with it, and inserting at its right edge does not. Two points conflict when they are equal.

Admission. A patch arrives as a callback that derives $[s, e)$ and replacement bytes x from the current file. It is admitted only if (i) every observation of the scope, on every file, is unexpired and byte-equal to the current contents, and every file it names still exists; and (ii) the valid observations of a single receipt of the scope cover $[s, e)$. Failing (i) refuses with the list of regions to reread and no file content; failing (ii) refuses with a read-required code.

Rebase. Committing a patch with $\delta = |x| - (e - s)$ transforms every observation of the resource (Figure 1). An observation that does not conflict with $[s, e)$ shifts by δ if it starts at or after e , and is otherwise unchanged. A conflicting observation splits into its left part $[a, s)$ and its shifted right part $[e + \delta, b + \delta)$, both still valid, and a middle $[s, s + |x|)$. The middle is marked stale for other scopes; for the writing scope it becomes an observation of x . For eight lockfile names (for example `package-lock.json` and `cargo.lock`), any patch invalidates every other scope’s observations of the file, which then need a whole-file reread.

Unmediated changes and blind writes. Whenever the authority materializes a resource whose presence, hash or length differs from its record, it increments the version and marks every observation of the resource stale. No coordinate transform is known for such a change, so only a whole-file read clears it. A whole-file write is *blind replacement*: it needs no read of its target and creates none, not even for the writer, though

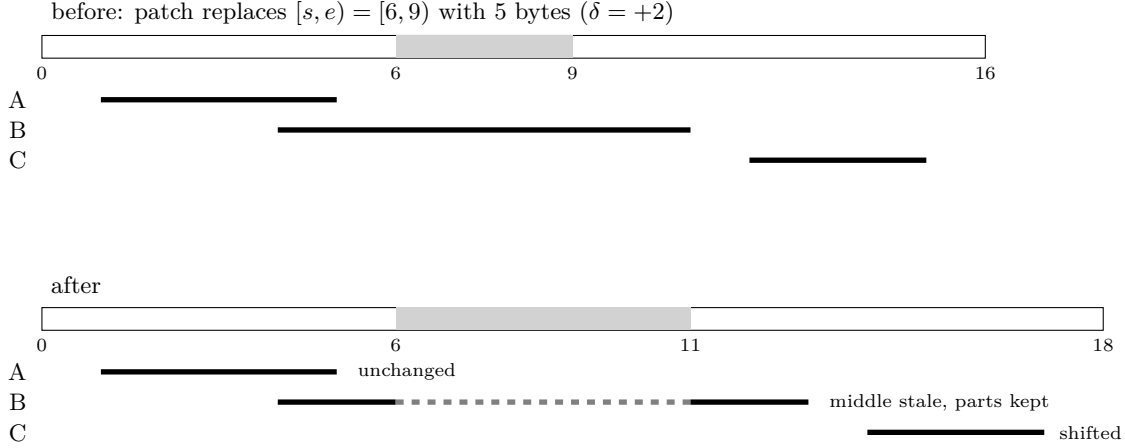


Figure 1: Rebasing of three other scopes’ observations when a patch replaces $[6, 9)$ with five bytes. Solid: valid; dashed: stale, reread required.

the writer’s read set is still validated. Once committed, it invalidates every observation of the resource, the writer’s included.

Leases. An observation lasts 60 minutes. When another scope holds a live observation of the same resource, a new one lasts 60 seconds, and every observation of that resource is capped at 60 seconds. An expired observation must be reread.

Publication, creation and recovery. Every operation runs inside one *lifetime lock*: an SQLite `BEGIN IMMEDIATE` transaction on a dedicated lock database [28, 29], held for the whole asynchronous operation. The operating system releases it if the holder dies, and nothing steals it by age. State lives in a second database with full synchronous commits, so a `PREPARED` record, carrying a SHA-256 checksum over the operation, is durable before publication while the lock is still held. The publisher replaces the file by rename [32] and receives a synchronous scope check to run immediately before it. The authority then re-materializes the file, confirms the after-image, and in one transaction applies the rebase or invalidation, increments the version and marks the operation `COMMITTED`. If the scope is revoked after commit, the call reports that the write committed, never that it was refused. Creating a missing file writes an exclusively created, fsynced sibling stage whose device, inode, digest and length are journaled in `PREPARED`, and publishes it by hard link, which fails rather than replace a file that appeared meanwhile [31]. Recovery runs before every admission, for the target and for every resource the scope has read. It validates each pending operation’s checksum, fields, scope and base version, then lets the bytes decide: the before-image marks it `ABORTED`, and the after-image (with the staged identity, for a creation) marks it `COMMITTED` with its effects applied. Any other state marks it `UNKNOWN`, which blocks the resource and the scopes that read it, but not unrelated work. A missing or corrupt store refuses; it is never treated as empty history.

4 Invariants

Write F_r for the current bytes of resource r and $F_r[a:b]$ for a slice. An observation is $o = \langle \sigma, r, [a, b), \beta, \tau, \kappa \rangle$: scope, resource, interval, observed bytes, expiry and stale reason ($\kappa = \perp$ if none). It is *valid* at time t when

$$\text{valid}(o, t) \iff \kappa = \perp \wedge \tau > t \wedge b \leq |F_r| \wedge F_r[a:b] = \beta.$$

The following hold by construction of the code paths named in Section 3, assuming correct adapters and SQLite’s transactional atomicity.

I1 Receipts. A receipt for $[a, b)$ of r at version v records $H(F_r[a:b])$ and $H(F_r)$ as materialized at v , and is durable before the bytes are released.

- I2 Admission-time validation.** An admitted patch of $[s, e]$ by σ passes a read-set check in which every observation is valid at validation time t_v , followed by a coverage check on the valid observations of one receipt of σ on r . Expiry is checked during admission; it is not rechecked as a publication deadline.
- I3 Invalidation.** Committing a write marks stale every other scope’s observation that conflicts with its span (for a whole-file write, every observation of the resource), in the same transaction that marks the operation COMMITTED.
- I4 Drift.** If a materialization of r differs from the recorded presence, hash or length, the version of r increases and every observation of r becomes stale until a whole-file read.
- I5 Journal.** PREPARED is durable before publication. COMMITTED implies the after-image, and for a creation the staged identity, was confirmed on disk. Recovery resolves a non-no-op PREPARED operation to ABORTED on the before-image, to COMMITTED on the after-image, or to UNKNOWN otherwise. A pending no-op commits without a new publication. An operation’s read-set effects and version increment are applied exactly when it becomes COMMITTED.

Proposition (no lost update between mediated patches). *If every write to r is mediated, a committed patch never overwrites a byte whose latest write its scope did not observe.* Sketch. Let patch P by σ replace $[s, e]$. By I2, admission establishes valid observations covering $[s, e]$, so $F_r[s:e]$ equals what σ observed at that check. The same authority lock excludes another mediated writer through publication; lease expiry alone does not change the bytes. Suppose a write W by another scope changed a byte in $[s, e]$ after σ observed it. By I3, W ’s commit marked σ ’s conflicting observation stale, which contradicts validity. So σ must have reread after W , and the byte’s latest write is one it observed. Bytes outside $[s, e]$ are untouched, and their observations are shifted, not invalidated. An unmediated change before P ’s admission is caught by I4, because admission materializes r . A change between the publisher’s final comparison and its rename is outside the proposition’s assumption.

The same argument, applied to observations on other files, gives read-set freshness. Provided every writer of every observed resource uses the same authority, the lock preserves the validated byte contents through publication. This covers an intervening mediated interface change, but it does not fence an unmediated writer that changes a dependency after that dependency was checked. A receipt’s lease can also expire after admission and before publication; the implementation has no final read-set expiry check. These are sketches of the mechanism, not machine-checked proofs.

5 Implementation

The authority is one 875-line CommonJS module for Node.js 22. It uses the built-in `node:sqlite` module [21], which Node still labels experimental; the warning appears in our logs. The data store has seven tables (metadata, scopes, resources, receipts, observations, operations and an append-only event log) at schema version 2, with an in-place migration from version 1 that preserves version-1 journal records and their checksums byte for byte. Refusals carry typed codes and repair lists but never file content. Callers supply four adapters: `materialize`, `publish`, and `prepareCreate` and `reconcileCreateStage` for creation.

In the engine, the authority backs the `repo.read_file`, `repo.write_file` and `repo.patch_file` tools and the paired-desktop `workspace.read`. The engine’s adapters read through a descriptor opened with `O_NOFOLLOW`, bounded in size, and compare device, inode, size and timestamps before and after the read. They refuse symbolic-link and hard-link aliases. On publication they compare the before-image, write and fsync a temporary, compare the before-image again, and rename under a whole-file lock. `repo.patch_file` derives its span as the unique occurrence of the caller’s old text in the current bytes. Each tool call also receives a one-shot invocation capability, and paired-desktop scopes are revoked on disconnect, timeout, key rotation and shutdown.

The standalone extraction keeps the engine’s directory layout. The authority, its five suites and their three helper files are byte-identical to the engine (git blob hashes are listed in the repository). The two modules the authority requires are replaced by POSIX stand-ins: path admission reduces to the engine’s POSIX behavior, an absolute-path check and normalization, and the default state directory follows XDG conventions. Windows account confinement is not extracted, and the stand-in refuses to run on Windows.

Suite	Standalone 2026-09-11	Engine c34cab8c	Exercises
byte-authority	17/17	17/17	receipts, admission, rebase, leases, lock-files, revocation
byte-authority-recovery	8/8	8/8	failed publication, quarantine, store loss and corruption
byte-authority-process	10/10	10/10	real processes: contention, exit at crash points
byte-authority-write	15/15	15/15	blind writes, creation, dependencies, final guards
byte-authority-write-recovery	18/18	18/18	creation recovery, identity checks, migration
repo-byte-transport	–	27/27	registry and MCP dispatch, text encodings
remote-byte-scope	–	10/10	paired-desktop scopes and revocation
fra-byte-mediation	–	12/12	remote workspace reads through the same authority
Total	68/68	117/117	

Table 1: Passing tests over tests run, one file per process. Engine runs are at engine commit c34cab8c, whose authority and suites are byte-identical to the extraction. Sources: `data/standalone/`, `data/engine-runs/`, `data/results.csv`.

Probe	Class	Reproduced behavior
P1	liveness	A deleted file in a scope’s read set blocks that scope’s later write to an unrelated file.
P2	liveness	A second reader caps an unchanged observation’s lease at 60s; after it lapses the first reader must reread.
P3	lost update	A blind whole-file write of bytes read outside mediation erases a peer’s committed patch.
P4	semantics	Two byte-disjoint patches turn “1,1” into “0,0”; both commit.
P5	identity	A new scope has no predecessor read set, so a resumed agent’s carried-over context is unchecked.
P7	retention	A committed whole-file payload is still in the store after its scope closes.

Table 2: Design probes. P6 of the original set tested an unrelated module and is omitted. Sources: `probes/`, `data/standalone/design-probes.tap`, `data/engine-runs/2026-09-07/design-probes.tap`.

6 Evaluation

Test suites. Table 1 lists every suite. The five core suites exercise the authority through real SQLite databases and real files. The operations include two concurrent operating-system processes making disjoint edits with shifted coordinates, a competing process blocked on the held lock, and process exit before publication, after publication and between link and unlink. They also include injected publisher failures, deleted and corrupted stores, revocation in an asynchronous gap, racing creators, extra hard-link aliases, equal-content file substitution, and version-1 migration. The three transport suites run only in the engine; they exercise its tool registry, MCP dispatch and paired-desktop transport. All results come from one file per process. Standalone runs used Node.js 22.19.0 on Linux x64.

Design probes. A design review wrote seven probes against the engine, and six of them concern the authority (Table 2). A probe passes when the limitation it names is reproduced. Each passed against the engine on 2026-09-07 and again in the standalone extraction. The probes are constructed demonstrations of mechanism; they say nothing about how often these cases arise in practice.

Admission-boundary review. Two additional controlled checks reproduce the boundaries of I2. Advancing the injected clock past the read lease after admission still permits publication; changing an observed dependency through an unmediated writer after read-set validation also permits publication on the target. These checks document limits, not desired application behavior, and are separate from the 68 core tests and six original probes. Source and captured output are in `review/`. No engine code was changed by this review.

Not measured. We did not measure latency or throughput, refusal or conflict rates with real agents, whether agents reconcile correctly after a refusal, durability under power loss, or behavior on Windows. The authority serializes every operation through one cross-process transaction, and its many-agent throughput is unknown.

7 Deployment experience

In the inspected ToolsEnabled engine snapshot (source commit `c34cab8c`; engine package metadata 1.4.0), the authority was wired to the `repo.*` tools and the paired-desktop read path. The `repo.*` tools resolve paths against the engine’s own package root. In an installed build that root is the installed payload, and generic writes into it are refused. The agents in that deployment edited their working clones through `host.read_file`, `host.write_file` and `host.exec`. The host file provider takes a whole-file write lock and records an audit entry, but never consults the authority. The authority was therefore bypassed by that coding workflow. No production conflict dataset accompanies this reviewed workflow, and none is reported here. These source and per-suite receipts are not an installed-release acceptance receipt for the final 1.0.44 cut. A mediation layer protects only the write paths routed through it, and the paths these agents used were not.

Mediation of `host.read_file`, `host.write_file` and a new `host.patch_file` through the same authority has since been implemented on an engine branch (tip `3c40a8d4`). It is ready for repackaging but not yet deployed. It uses a store namespace separate from `repo.*`, prunes committed payloads at commit, and adds an off switch. Unlike `repo.write_file`, `host.write_file` replaces an existing file only when the writing scope currently observes all of it, so the P3 scenario refuses for host files. Its new 11-test suite covers these rules, retention, crash recovery and an end-to-end call through the engine’s MCP dispatch. All 11 fail at the base engine, the first because agent A’s stale write silently overwrites B’s; all 11 pass on the branch (`data/host-mediation-branch/`).

The branch also reports two measurements, taken on a machine that was running other work. First, over 100 interleaved iterations on a 100 KB file, a `host.read_file` plus `host.write_file` round trip through the engine’s tool dispatcher took 266.01 ms at the median and 334.26 ms at the 95th percentile with mediation, against 213.73 ms and 266.01 ms without it. Second, over 500 writes of 100 KB, the host store grew by 141,148,160 bytes when committed payloads were kept and by 4,931,584 bytes when they were pruned at commit.

8 Related work

Concurrency control and isolation. Optimistic concurrency control records a transaction’s read set during execution and validates it at commit [15]. The authority applies that rule to file bytes, with the read set built from mediated tool calls. Predicate locks [6] protect sets defined by conditions; byte intervals are range predicates over offsets, used here for validation rather than locking. The lost update, read skew and write skew are characterized by Berenson et al. [2], generalized by Adya [1], and addressed for snapshot isolation by Fekete et al. [7]. Leases bound how long cached state may be trusted [9], as observation expiry does here.

Conditional requests. HTTP makes a write conditional on a validator such as an entity tag through `If-Match` [8]. A server can require such preconditions with status 428 [22], and ETags with unreserved checkout were proposed specifically against the lost update in web authoring [20]. Byte-interval mediation is a conditional write whose precondition is the writer’s whole read set at byte granularity. The server records the read set as reads happen, instead of trusting a validator presented by the client.

Operational transformation, CRDTs and merging. Operational transformation rewrites concurrent operations so that replicas converge [5, 30]. CRDTs converge by construction [14, 26]. Three-way merge integrates divergent versions after the fact [12, 18]. Rebase is a position transform of the same kind as in operational transformation, applied to read intervals rather than to operations. Where intervals conflict, the authority refuses instead of transforming, because an agent’s premise, not only its text, may have changed. Conflicts that survive a clean textual merge are studied by Brun et al. [3], and Sousa et al. [27] verify semantic conflict freedom of merges. Probe P4 is such a conflict.

Intent logging and crash consistency. Write-ahead logging with redo and undo is the foundation of transactional recovery [19]. Application-level crash consistency on file systems depends on fsync and rename behavior that applications often get wrong [23]. The authority’s journal is an intent log for a single file publication, and recovery decides between redo and abort by comparing the bytes on disk with the journaled before-image and after-image.

Agents and change impact. Work on LLM software engineering, from benchmarks to agent systems [11, 33, 35–37], mostly evaluates one agent working on a repository at a time. Multi-agent frameworks [10, 25, 34] organize agents by roles, conversations and hand-offs, and a taxonomy of multi-agent failures [4] includes coordination breakdowns. This paper concerns the layer below: what happens when two agents’ edits to the same file interleave. Three recent preprints address that layer directly. CooperBench evaluates coding agents that must coordinate separate features in one repository [13]. STORM mediates agents’ interactions with a shared workspace, so that each works on a consistent view and conflicting edits are detected and resolved at write time [16]. CoAgent argues that locks block long inference and that optimistic abort-and-retry discards work. It fixes a serialization order at launch, applies writes speculatively in place, and has affected agents repair their plans [17]. Byte-interval mediation refuses a stale write and names the regions to reread, which leaves reconciliation to the agent; we have not measured what that costs in agent work. Co-change mining predicts which files change together [38]. The companion Blast-Radius project [24] designs an estimator for which regions a change may invalidate where no read was recorded. Byte-interval mediation is the exact tier beneath that estimator: where a read was recorded, the answer is a lookup.

9 Limitations and future work

- **Semantic dependence.** Receipts show what was read, not what the work depends on (P4). A next step is *semantic dependency witnesses*: evidence recorded beside receipts, such as tests executed, symbols resolved or contracts consulted, against which a write can be checked.
- **Coverage.** Unmediated writers, including shell commands, are detected only when next observed, and a change landing between the final comparison and the rename is not caught. Host file mediation (Section 7) is not yet deployed.
- **Blind writes** of bytes obtained outside mediation can erase a peer’s patch (P3).
- **Scope continuity.** Scopes are not bound to launch, lane or run records, so a resumed agent does not inherit or revalidate its predecessor’s read set (P5).
- **Liveness.** A deleted dependency blocks its scope until the file is restored and reread (P1), and contention cuts leases to 60 seconds (P2). The repair budget with escalation that the design specified is not built.
- **Retention.** The extracted core retains committed payloads (P7). The separately reported host-mediation branch adds pruning; that does not change the behavior of this extraction.
- **Scale and platform.** There is a 512 KiB limit per file and a single serializing transaction. Throughput is unmeasured, recovery covers process crashes rather than power loss, and the extraction runs on POSIX only.

Author contributions and AI assistance

Joshua Pinckard conceived the project, defined its objectives and requirements, directed the autonomous agent workflows, selected and evaluated outputs, and assumes responsibility for the research methodology and conclusions. AI agents generated substantial portions of the implementation and written drafts.

Availability

Code, tests, probes and data: <https://github.com/JoshuaPinckard/byte-interval-mediation>, MIT License.

References

- [1] Atul Adya. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*. PhD thesis, Massachusetts Institute of Technology, March 1999. URL <https://hdl.handle.net/1721.1/149899>. Also available as MIT technical report MIT-LCS-TR-786.
- [2] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. A critique of ANSI SQL isolation levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data (SIGMOD ’95)*, pages 1–10. ACM Press, 1995. doi: 10.1145/223784.223785.
- [3] Yuriy Brun, Reid Holmes, Michael D. Ernst, and David Notkin. Proactive detection of collaboration conflicts. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE ’11)*, pages 168–178. ACM, 2011. doi: 10.1145/2025113.2025139.
- [4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? arXiv preprint arXiv:2503.13657, 2025. URL <https://arxiv.org/abs/2503.13657>.
- [5] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data (SIGMOD ’89)*, pages 399–407. ACM Press, 1989. doi: 10.1145/67544.66963.
- [6] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger. The notions of consistency and predicate locks in a database system. *Communications of the ACM*, 19(11):624–633, November 1976. doi: 10.1145/360363.360369.
- [7] Alan Fekete, Dimitrios Liarokapis, Elizabeth O’Neil, Patrick O’Neil, and Dennis Shasha. Making snapshot isolation serializable. *ACM Transactions on Database Systems*, 30(2):492–528, June 2005. doi: 10.1145/1071610.1071615.
- [8] R. Fielding, M. Nottingham, and J. Reschke. HTTP semantics. RFC 9110, RFC Editor, <https://www.rfc-editor.org/info/rfc9110>, June 2022. Internet Standard, STD 97.
- [9] C. Gray and D. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP ’89)*, pages 202–210. ACM, 1989. doi: 10.1145/74850.74870.
- [10] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, pages 23247–23275, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, pages 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.

- [12] Sanjeev Khanna, Keshav Kunal, and Benjamin C. Pierce. A formal investigation of Diff3. In *FSTTCS 2007: Foundations of Software Technology and Theoretical Computer Science*, volume 4855 of *Lecture Notes in Computer Science*, pages 485–496. Springer Berlin Heidelberg, 2007. doi: 10.1007/978-3-540-77050-3_40.
- [13] Arpandeeep Khatua, Hao Zhu, Peter Tran, Arya Prabhudesai, Frederic Sadrieh, Johann K. Lieberwirth, Xinkai Yu, Yicheng Fu, Michael J. Ryan, Jiaxin Pei, and Diyi Yang. CooperBench: Why coding agents cannot be your teammates yet. arXiv preprint arXiv:2601.13295, 2026. URL <https://arxiv.org/abs/2601.13295>.
- [14] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. *IEEE Transactions on Parallel and Distributed Systems*, 28(10):2733–2746, October 2017. doi: 10.1109/TPDS.2017.2697382.
- [15] H. T. Kung and John T. Robinson. On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2):213–226, June 1981. doi: 10.1145/319566.319567.
- [16] Mengyang Liu, Taozhi Chen, Zhenhua Xu, Xue Jiang, and Yihong Dong. Multi-agent collaboration with state management. arXiv preprint arXiv:2605.20563, 2026. URL <https://arxiv.org/abs/2605.20563>.
- [17] Hongtao Lyu, Dingyan Zhang, Mingyu Wu, Xingda Wei, and Haibo Chen. CoAgent: Concurrency control for multi-agent systems. arXiv preprint arXiv:2606.15376, 2026. URL <https://arxiv.org/abs/2606.15376>.
- [18] T. Mens. A state-of-the-art survey on software merging. *IEEE Transactions on Software Engineering*, 28(5): 449–462, May 2002. doi: 10.1109/TSE.2002.1000449.
- [19] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, March 1992. doi: 10.1145/128765.128770.
- [20] Henrik Frystyk Nielsen and Daniel LaLiberte. Editing the Web: Detecting the lost update problem using unreserved checkout. W3C Note, May 1999. URL <https://www.w3.org/1999/04/Editing/>. Dated 10 May 1999; this version: <https://www.w3.org/1999/04/Editing/01>. Accessed 2026-09-11.
- [21] Node.js. SQLite (node:sqlite). Node.js v22.23.2 Documentation, 2026. URL <https://nodejs.org/docs/latest-v22.x/api/sqlite.html>. Added in v22.5.0; no longer behind the `--experimental-sqlite` flag since v22.13.0 (still marked Stability 1.1, active development). Accessed 2026-09-11.
- [22] M. Nottingham and R. Fielding. Additional HTTP status codes. RFC 6585, RFC Editor, <https://www.rfc-editor.org/info/rfc6585>, April 2012.
- [23] Thanumalayan Sankaranarayanan Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 433–448, Broomfield, CO, October 2014. USENIX Association. ISBN 978-1-931971-16-4. URL <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai>.
- [24] Joshua Pinckard. Blast-Radius. Unpublished research repository, 2026. Software, Apache License 2.0.
- [25] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.810.
- [26] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer Berlin Heidelberg, 2011. doi: 10.1007/978-3-642-24550-3_29.
- [27] Marcelo Sousa, Isil Dillig, and Shuvendu K. Lahiri. Verified three-way program merge. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):165:1–165:29, 2018. doi: 10.1145/3276535.
- [28] SQLite. File locking and concurrency in SQLite version 3. SQLite documentation, 2025. URL <https://www.sqlite.org/lockingv3.html>. Page last updated 2025-05-31. Accessed 2026-09-11.
- [29] SQLite. Transaction. SQLite documentation, 2026. URL https://www.sqlite.org/lang_transaction.html. Page last updated 2026-02-18. Accessed 2026-09-11.

- [30] Chengzheng Sun, Xiaohua Jia, Yanchun Zhang, Yun Yang, and David Chen. Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 5(1):63–108, March 1998. doi: 10.1145/274444.274447.
- [31] The IEEE and The Open Group. link, linkat — hard link one file to another file. The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024, 2024. URL <https://pubs.opengroup.org/onlinepubs/9799919799/functions/link.html>. Accessed 2026-09-11.
- [32] The IEEE and The Open Group. rename, renameat — rename file. The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024, 2024. URL <https://pubs.opengroup.org/onlinepubs/9799919799/functions/rename.html>. Accessed 2026-09-11.
- [33] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, pages 65882–65919, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html.
- [34] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [35] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. arXiv preprint arXiv:2407.01489, 2024. URL <https://arxiv.org/abs/2407.01489>.
- [36] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652, 2024. doi: 10.52202/079017-1601.
- [37] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA ’24)*, pages 1592–1604. ACM, 2024. doi: 10.1145/3650212.3680384.
- [38] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. Mining version histories to guide software changes. *IEEE Transactions on Software Engineering*, 31(6):429–445, June 2005. doi: 10.1109/TSE.2005.72.